Midterm Topics: PHIL 121

Exam: October 14, 2026, in class

Format. Closed book, closed notes, no computer. You write code by hand. You will be asked to read code and give the output, find bugs in code, and write a short functions. Covers everything from Weeks 1 - 6.

Topics

Variables and types

- An assignment runs once, using the values at that moment. `y = x + 1` does not update when `x` changes later.
- `a = b` then `b = a` does *not* swap. To swap, write `a, b = b, a`.
- `+` joins strings (`"3" + "3"` is `"33"`) but adds numbers (`3 + 3` is `6`). You cannot `+` a string and a number; convert first with `str(...)` or `int(...)`.
- `print(a, b)` prints the values separated by a space.

Names

- Valid names use letters, digits, and underscores, and cannot start with a digit. No hyphens, spaces, or symbols, and not a keyword like `for`.
- Valid: `total`, `my_list`, `_count`. Invalid: `2nd`, `my-list`, `my list`, `for`.

Functions

- `def name(params):` then an indented body; `return` sends a value back.
- A function with no `return` gives back `None`. `print` is not `return`: `y = f(5)` is `None` if `f` only prints.
- Default values: `def order(item, size="small")`. Keyword arguments: `order("tea", size="large")`. Positional arguments must come before keyword arguments.

Booleans and truthiness

- `and`, `or`, `not`. `not` binds tightest, then `and`, then `or`.
- Truthy vs falsy: `0`, `""`, `[]`, and `None` are falsy; most other values are truthy.
- Watch the trap: `if x == "a" or "b":` is `(x == "a") or ("b")`, and `"b"` is always truthy. Write `x == "a" or x == "b"`.
- Conditional expression: `big if n > 10 else small`.

Conditionals

- `if` / `elif` / `else`. Order matters: the first true branch wins, so put the most specific test first.
- A separate `if` (not `elif`) can run in addition to an earlier one.

`for` loops

- `range(n)` is `0..n-1`; `range(a, b)` is `a..b-1`; `range(a, b, step)` counts by `step`.
- You can loop over a string (`for ch in word`) or a list (`for item in items`).
- Counter/accumulator pattern: start a variable at 0, update it each pass.

`while` loops

- Repeats while the condition is true. Make sure something inside changes, or it never stops.
- "Keep going until" pattern, e.g. play until someone reaches 3 wins: `while player_wins < 3 and computer_wins < 3:`.

Strings

- `len(s)`, index with `s[0]` and `s[-1]`, `"a" in "cat"`, and loop with `for ch in s`.

Lists

- Make one with `[...]`; `len(xs)`; index `xs[0]`, `xs[-1]`; a list can hold any types.
- Lists are mutable: `xs[1] = "new"`. Check membership with `"c" in xs`.
- Slices: `xs[1:3]`, `xs[:2]`, `xs[2:]`.
- `xs.append(v)` adds to the end, `xs.pop(i)` removes and returns, `xs.remove(v)` deletes by value. These change the list and return `None`: never write `xs = xs.append(v)`.
- `list("ab")` is `['a', 'b']`; `"a b".split()` is `['a', 'b']`; `"-".join(["a", "b"])` is `"a-b"`.
- Build with a loop: start `result = []`, then `result.append(...)`.
- Comprehension: `[n * n for n in range(5)]`, with a filter `[n for n in xs if n > 0]`.
- `sum(xs)`, `min(xs)`, `max(xs)`, `sorted(xs)` (returns a new sorted list).

Tuples

- Like a list but written with parentheses and *cannot* be changed: `p = (3, 4)`.
- Unpack with `x, y = p`. A function can return several values as a tuple: `return low, high`.

1 Practice Problems

1. What is the output of the following code?

```
xs = [1, 2, 3]
ys = xs
ys.append(4)
print(xs)
```

2. What is the output of the following code?

```
nums = [10, 20, 30, 40, 50]
print(nums[1:4])
print(nums[-2:])
```

3. What is the output of the following code?

```
words = ["hi", "hello", "hey", "yo"]
print([w for w in words if len(w) <= 2])
```

4. What is the output of the following code?

```
grid = [[1, 2], [3, 4]]
total = 0
for row in grid:
    for x in row:
        total = total + x
print(total)
```

5. What is the output of the following code?

```
sentence = "the quick brown fox"
words = sentence.split()
print(len(words), words[0], words[-1])
```

6. What is the output of the following code?

```
s = "python"
print(s[0], s[-1], s[1:4], len(s))
```

7. What is the output of the following code?

```
nums = [3, 1, 2]
print(sorted(nums))
print(nums)
nums.sort()
print(nums)
```

8. What is the output of the following code?

```
def min_max(nums):
    return min(nums), max(nums)

lo, hi = min_max([5, 2, 9, 1])
print(lo, hi)
```

9. What is the output of the following code?

```
result = []
for i in range(1, 4):
    result.append(i * 10)
print(result)
```

10. What is the output of the following code?

```
fruits = ["apple", "pear"]
print("apple" in fruits, "grape" in fruits, len(fruits) == 2)
```

11. What is the output of the following code?

```
xs = [1, 2, 3, 4]
xs.pop(0)
xs.remove(3)
print(xs)
```

12. What is the output of the following code?

```
print("ab" * 3)
print([1, 2] * 2)
print(3 * "7")
```

13. What is the output of the following code?

```
print(int("10") + 5)
print(str(10) + "5")
print(int("3") * "ab")
```

14. What is the output of the following code?

```
print(8 / 2)
print(len("python"))
print(len([3, 6, 9]))
```

15. What is the output of the following code?

```
for value in [0, "", [], None, "hi", [1], 5]:
    if value:
        print("truthy")
    else:
        print("falsy")
```

16. What is the output of the following code?

```
items = []
if items:
    print("yes")
else:
    print("no")
print(len(items))
```

17. What is the output of the following code?

```
print(0 or 10)
print("" or "default")
print("hi" or "default")
```

18. What is the output of the following code?

```
def sign(n):
    if n > 0:
        return "pos"
    elif n < 0:
        return "neg"

print(sign(5))
print(sign(0))
```

19. What is the output of the following code?

```
def first(items):
    for x in items:
        return x

print(first([10, 20]))
print(first([]))
```

20. What is the output of the following code?

```
def winner(player, computer):
    if player == computer:
        return "tie"
    elif (player == "rock" and computer == "scissors") or \
         (player == "paper" and computer == "rock") or \
         (player == "scissors" and computer == "paper"):
        return "player"
    else:
        return "computer"

print(winner("rock", "scissors"), winner("paper", "paper"), winner("
    scissors", "rock"))
```

21. What is the output of the following code?

```
def cycle(round_number, last_computer_move):
    leftover = round_number % 3
    if leftover == 1:
        return "rock"
    elif leftover == 2:
        return "scissors"
    else:
        return "paper"

for r in range(1, 5):
    print(cycle(r, None))
```

22. This strategy is provided in Project 1. In one or two sentences, explain what it does.

```
def by_round(round_number, last_computer_move):
    if round_number % 2 == 0:
        if random.randint(1, 2) == 1:
            return "rock"
        else:
            return "paper"
    else:
        if random.randint(1, 2) == 1:
            return "paper"
        else:
            return "scissors"
```

23. This function is provided in Project 1. Explain what it does and what it returns.

```
def validate_input(player):
    if player == "rock" or player == "paper" or player == "scissors":
        return True
    else:
        print("Invalid input. Please enter rock, paper, or scissors.")
        return False
```

24. This code will not run. Explain the error and say how to fix it.

```
def greet(name)
    return "Hi " + name
```

25. This code will not run. Explain the error and say how to fix it.

```
x = 5
if x > 3
    print("big")
```

26. This code will not run. Explain the error and say how to fix it.

```
print("hello)
```

27. This code will not run. Explain the error and say how to fix it.

```
x = 5
if x = 5:
    print("five")
```

28. This code will not run. Explain the error and say how to fix it.

```
def f(x):
return x + 1
```

29. This code runs, but there is a bug. The expected output is `[1, 2, 3]`, but the actual output is `None`. Explain how to fix the code.

```
nums = [3, 1, 2]
nums = nums.sort()
print(nums)
```

30. This code runs, but there is a bug. The expected output is `6`, but the actual output is `3`. Explain how to fix the code.

```
def total(numbers):
    for x in numbers:
        s = 0
        s = s + x
    return s

print(total([1, 2, 3]))
```

31. This code runs, but there is a bug. The expected output is `['eggs', 'milk']`, but the actual output is `['eggs']`. Explain how to fix the code.

```
def add_item(items, x):
    items = items + [x]

shopping = ["eggs"]
add_item(shopping, "milk")
print(shopping)
```

32. This code runs, but there is a bug. The expected output is `False`, but the actual output is `True`. Explain how to fix the code.

```
def is_vowel(ch):
    if ch == "a" or "e" or "i" or "o" or "u":
        return True
    return False

print(is_vowel("z"))
```

33. This code runs, but there is a bug. The expected output is `3`, but the actual output is `1`. Explain how to fix the code.

```
def count_positives(numbers):
    count = 0
    for x in numbers:
        if x > 0:
            count = count + 1
        return count

print(count_positives([1, -2, 3, 4]))
```

34. This code runs, but there is a bug. The expected output is the numbers 1 through 5, but the actual output is the numbers 1 through 4 (5 is missing). Explain how to fix the code.

```
# should print the numbers 1 through 5
for i in range(1, 5):
    print(i)
```

35. This code runs, but there is a bug. The expected output is 5, 4, 3, 2, 1 each on its own line, but the actual output is it prints `5` over and over and never stops. Explain how to fix the code.

```
n = 5
while n > 0:
    print(n)
```